

Módulo 3: Caso Aplicado

Diseño de un agente AFP desde Decision Science hasta RAG

2026-04-30

Enlaces

-  segana@fen.uchile.cl
 -  <https://segana.netlify.app>
 -  <https://www.linkedin.com/in/sebastian-egana-santibanez/>
 -  <https://github.com/sebaegana>
-

Caso aplicado

Asistente AFP que responde consultas de afiliados

Cómo diseñar un agente útil, seguro y realista para responder preguntas sobre pensiones

Objetivo de la actividad

- Diseñar un agente que responda consultas previsionales usando información oficial
 - Aplicar principios de Decision Science antes de construir la solución
 - Entender cómo se combinan LLM, RAG, workflow y control humano
 - Identificar límites, riesgos y decisiones de diseño en un caso realista
-

Contexto del caso

Caso: “Queremos un agente que responda preguntas sobre pensiones usando información oficial.”

El desafío organizacional

Una AFP quiere mejorar atención a afiliados sin exponerlos a respuestas erróneas, ambiguas o riesgosas.

La promesa

Responder más rápido, más claro y con mejor acceso a información.

El riesgo

Dar una respuesta incorrecta en un tema altamente sensible.

Parte 1

Antes de construir: definir la decisión

La primera pregunta no es tecnológica.

Es esta:

¿Qué decisión organizacional estamos tratando de mejorar con este agente?

Decision Science aplicada al caso

Antes de hablar de LLM o RAG, conviene estructurar el problema:

1. ¿Qué necesidad resuelve?
 2. ¿Qué alternativas existen?
 3. ¿Qué riesgos hay?
 4. ¿Qué nivel de autonomía es aceptable?
 5. ¿Qué decisiones deben quedar en manos humanas?
-

¿Qué problema resuelve este agente?

Posibles respuestas:

- reducir tiempos de atención
- mejorar consistencia en respuestas
- facilitar acceso a información compleja
- apoyar a ejecutivos humanos
- ampliar cobertura de atención básica

Idea clave

Un agente mal definido suele fallar aunque la tecnología sea buena.

Delimitar el alcance

No todo lo previsional debería responderse automáticamente.

Preguntas razonables para automatizar

- “¿Qué es el multifondo?”
- “¿Por qué puede bajar mi saldo?”
- “¿Qué significa esta sección de mi cartola?”
- “¿Cuándo puedo pensionarme según la regla general?”

Preguntas delicadas

- “¿Cuánto ganaré de pensión?”
 - “¿Qué me conviene hacer con mis ahorros?”
 - “¿Debo cambiarme de fondo ahora?”
-

Estructurar la decisión de diseño

Antes de construir, el equipo debe decidir:

- qué sí responde el agente
 - qué no responde
 - cuándo debe escalar a humano
 - qué fuentes puede usar
 - qué tono y nivel de claridad debe tener
-

Trade-offs del caso

- velocidad vs precisión
- autonomía vs control
- cobertura vs riesgo
- personalización vs privacidad

Pregunta útil

¿Preferimos responder más preguntas o responder menos, pero con más seguridad?

Sesgos y errores posibles

En este caso pueden aparecer:

- exceso de confianza del usuario
- sesgo de automatización
- falsa sensación de precisión
- sobreinterpretación de respuestas probabilísticas

Riesgo central

Que una respuesta “bien escrita” sea tomada como recomendación previsional válida.

Parte 2

Definir el agente

Una vez delimitado el problema, recién pasamos a la solución.

Definición simple

Un agente es un sistema que recibe una consulta, busca contexto y genera una respuesta siguiendo reglas.

Qué hará el agente

1. Recibe la pregunta del afiliado.
 2. Busca información relevante.
 3. Usa un LLM para redactar una respuesta clara.
 4. Devuelve una salida simple y comprensible.
-

Componentes del agente

LLM

Razona sobre la pregunta y redacta la respuesta.

RAG

Busca información en documentos como FAQ, cartola y normativa.

Output

Entrega una respuesta clara, simple y trazable.

Workflow del caso

Usuario → Agente

Agente → Busca información

Agente → LLM redacta respuesta

Agente → Entrega respuesta al afiliado

Lectura pedagógica

Prompt + RAG + automatización.

Fuentes posibles del agente

- cartola simulada del afiliado
- FAQ institucional
- normativa previsional resumida
- glosario de conceptos

Regla de diseño

Mientras más sensible la pregunta, más importante la calidad de la fuente.

Parte 3

Herramientas sin código para la actividad

Opción 1: ChatGPT con archivos

La más simple y potente para clase.

Opción 2: Poe o Flowise

Permiten armar bots personalizados y flujos simples.

Opción 3: Make o n8n

Permiten mostrar visualmente el workflow completo.

Opción recomendada para clase

ChatGPT con archivos

Qué hacen los estudiantes:

- suben PDF con FAQ AFP
- suben cartola simulada
- hacen preguntas como usuarios
- observan cómo responde el sistema

Idea clave

Esto ya funciona como un mini agente con RAG.

Parte 4

Actividad práctica

Desafío

Construyan un agente que responda preguntas de afiliados.

Paso 1: entregar contexto

Cada equipo recibe:

- PDF con FAQ AFP
- ejemplo de cartola
- 3 o 4 preguntas tipo cliente

Ejemplos

- “¿Por qué bajó mi saldo?”
 - “¿Cuándo me puedo pensionar?”
 - “¿Qué es el multifondo?”
-

Paso 2: usar la herramienta

Con ChatGPT:

- suben los documentos
 - prueban preguntas
 - observan qué responde
-

Paso 3: iterar

Aquí ocurre el aprendizaje real.

Preguntas guía

- ¿Responde bien?
 - ¿Se equivoca?
 - ¿Le falta contexto?
 - ¿Responde demasiado técnico?
 - ¿Podría inducir a error?
-

Qué deberían descubrir

- límites del LLM
 - importancia del contexto
 - necesidad de reglas claras
 - valor del diseño del agente
 - necesidad de control en casos sensibles
-

Paso 4: subirlo a “agente real”

Después de probar manualmente, introducimos la idea de automatización.

Lo que hicieron manualmente

Un agente real podría hacerlo de forma automática y escalable.

Flujo

Usuario → Agente
↓
Busca info con RAG
↓
LLM responde

Paso 5: poner nombre a lo vivido

Ahora sí, los conceptos tienen significado real:

- **LLM**: lo que redactó la respuesta
 - **RAG**: los documentos que aportaron contexto
 - **Agente**: la automatización del flujo
 - **Workflow**: los pasos que sigue el sistema
-

Parte 5

Hacerlo fallar a propósito

La parte más potente del ejercicio es esta:

forzar un caso donde el sistema no debería responder con seguridad

Pregunta tricky

“¿Cuánto ganaré de pensión?”

¿Qué puede pasar?

- el modelo generaliza
 - inventa supuestos
 - entrega una respuesta demasiado segura
 - confunde orientación con recomendación
-

Discusión posterior

Aquí aparecen temas centrales:

- riesgo reputacional
 - mala decisión para el cliente
 - falsa confianza
 - necesidad de escalamiento a humano
 - importancia de límites explícitos
-

Qué debería hacer un agente bien diseñado

Ante una pregunta de alto riesgo, debería:

- reconocer el límite
- no inventar cifras
- explicar que requiere más antecedentes
- derivar a un canal humano o simulador formal

Idea clave

Un buen agente no es el que siempre responde.
Es el que sabe cuándo no debe responder.

Parte 6

Cierre estratégico

Pregunta final para la clase:

¿Confiarían en este agente para hablar con clientes reales?

Conexión con gobernanza y ética

Esta pregunta abre la conversación sobre:

- gobernanza
 - ética
 - accountability
 - diseño organizacional
 - supervisión humana
-

Aprendizaje clave para estudiantes

No necesitan saber programar para diseñar agentes útiles.

Pero sí necesitan saber:

- estructurar problemas
 - delimitar decisiones
 - identificar riesgos
 - definir criterios de calidad
 - diseñar procesos con control
-

Mensaje final

Un agente no se diseña empezando por la tecnología.

Se diseña empezando por:

- la decisión que queremos mejorar
- el riesgo que queremos controlar
- el proceso que queremos rediseñar

Después viene la arquitectura.

Variación opcional

Caso alternativo: agente analista de inversiones

Input

Tabla simple de precios o indicadores.

Tarea

- resumir
- detectar tendencia
- sugerir interpretación

Misma lógica

LLM + contexto + workflow + control.

Referencias sugeridas

- Materiales del curso sobre LLM, RAG, agentes y gobernanza
- Documentación de OpenAI sobre file search y prompting
- Materiales introductorios de Decision Science y sesgos cognitivos